FreightFlow PRO — Complete Master Build Prompt
(Full Architecture, Business Rules, Schema, Seed Data & Starter Code)

ROLE
 j
You are the Lead Software Architect and Senior Full-Stack Engineer for **FreightFlow PRO**, a real commercial desktop application for a freight forwarding company based in Pakistan (export-focused, sea freight, FCL/LCL). This replaces the company's existing workflow of 10+ Excel sheets and will be used daily by real office staff. Every decision must be treated as production software — not a prototype, not a tutorial.

Core principle: `Job_Number` is the master key. Everything (rates, containers, documents, taxes) links to one Job_Number, entered once.

Rules for every module you build, without exception:
- Explain what you're building and why it's needed before writing code.
- State exactly which files will be created or modified.
- Deliver production-quality, fully implemented code — no placeholder logic, no `// TODO`, no skipped validation or error handling.
- Keep code modular and maintainable; never rewrite unrelated modules while building a new one.
- Ensure clean integration with everything already built.
- Do not proceed to the next phase until I explicitly approve the current one.

---

1. THE PROBLEM

The company currently duplicates the same data (clients, containers, rates) across multiple Excel sheets per shipment. This causes typos, wastes time (a job takes ~45 minutes in Excel), and looks unprofessional. FreightFlow PRO lets staff enter data once against a Job_Number and generate professional documents with one click, cutting job entry time to ~5 minutes.

2. SCOPE

In scope: job/shipment lifecycle management (booked → sailed → delivered → closed/cancelled), master data (clients, ports, commodities, container types), buying/selling rate tracking in USD or PKR, currency-aware profit calculation, ZKT/KHRT tax calculation, PDF generation (Bill of Lading, Invoice, Booking Note, CRO

Request), GPSHT and JOBGP reports with Excel export, packaged as a Windows desktop app via Electron.

Explicitly out of scope for v1: auto-forwarding, WhatsApp/fax integration, mobile app, multi-user roles/permissions, complex analytics beyond Excel export, multi-office support, cloud hosting (data stays local), file attachments (schema must not block adding this later; do not build it now).

3. TECHNOLOGY STACK

Component	Technology	Why
Database	SQLite via **better-sqlite3**	Single file, no server, synchronous API — required for safe atomic multi-table writes (job+containers, profit+tax generation)
Backend	Node.js + Express	Fast, simple, JS everywhere
Frontend	HTML + CSS + Vanilla JavaScript, no framework	Lightweight, no build step, easy for future non-specialist maintainers
Desktop	Electron	Wraps the web app into a Windows `.exe`
PDF	jsPDF + jspdf-autotable	Local PDF generation, no external service dependency
Excel export	SheetJS (xlsx)	Report export
Packaging	electron-builder	Produces the NSIS installer

Important build note: `better-sqlite3` is a native module requiring compilation. On a machine with normal internet access, `npm install` handles this automatically via prebuilt binaries. If building in a network-restricted sandbox with no access to prebuilt binaries, you may temporarily test logic against Node's built-in `node:sqlite` (`DatabaseSync`, available Node 22.5+) using a thin compatibility shim — but the shipped `package.json` dependency and all production code must target `better-sqlite3`, since Electron packaging requires it (via `electron-rebuild`) and `node:sqlite` is not embedded in Electron's older bundled Node runtime.

---

4. IMPROVEMENTS ALREADY IDENTIFIED — MUST BE INCORPORATED

These were found during architecture review of an earlier draft and are **mandatory fixes**, not optional suggestions:

1. **Currency-unsafe profit calculation (critical bug in earlier draft).** Never sum `rates.amount` directly across rows without checking `currency` — a job can have both

```
USD and PKR rates, and summing them raw adds dollars to rupees. Every rate row must be converted to a common base currency (USD) individually before aggregation.
2. **Exchange rate must not be a static global config value.** Store it in an editable `settings` table, and **lock it onto the job** (`exchange_rate_locked`) the first time profit/tax is calculated for that job. Never let editing the global rate retroactively change historical job numbers.
3. **`bl_data` must not duplicate job data.** Only BL-specific fields belong there (vessel, voyage, ports of loading/discharge/delivery, freight terms, free days, issued date). Shipper, consignee, notify parties, commodity, packages, and weight must be read live from `jobs`/`clients` at print time.
4. **Add Port of Loading (POL).** The earlier draft only had POD. Export operations need both origin and destination ports on the job and on the BL.
5. **Add job type / direction fields.** `job_type` (FCL/LCL) and `direction` (EXPORT/IMPORT) so the schema isn't hardcoded to one mode even though v1 usage is primarily export/FCL.
6. **No hard deletes on financial data.** Jobs must be soft-deleted/archived (`is_archived` flag), never truly removed once any rate is marked PAID or any document generated. Master data uses `is_active` toggling rather than deletion where historically referenced.
7. **Add a `CANCELLED` job status.** The earlier draft's status enum (`BOOKED`/`SAILED`/`DELIVERED`/`CLOSED`) was missing this — real operations always have cancelled bookings.
8. **Add `created_at`/`updated_at` timestamps to every table**, auto-maintained via `AFTER UPDATE` triggers — never trust application code to remember to set these.
9. **Job number generation must be atomic.** Replace "read `MAX(job_number)`, then +1" with a dedicated `job_number_sequence` table (one row per year) incremented inside a transaction. This eliminates a real race condition.
10. **Automated local backups.** SQLite is a single file — one accidental delete or corruption event loses all financial records. Electron packaging must include daily rotating backups (keep last 30) of the database file, triggered on app close at minimum.
11. **Tax percentages (ZKT 2.5%, KHRT 7.5%) must be configurable**, stored in `settings`, not hardcoded in application logic.
12. **Add indexes on every foreign key column** and on columns used in dashboard search/filtering (`jobs.status`, `jobs.is_archived`, `jobs.created_date`, `clients.type`, `clients.is_active`), so performance holds up as job volume grows into the thousands.

---

## 5. BUSINESS RULES (source of truth)
```

1. Profit = Σ(SELLING rates, each row converted to USD using the job's locked exchange rate) − Σ(BUYING rates, same conversion).
2. Exchange rate locks onto the job at first profit/tax generation; regenerating requires an explicit user action.
3. Taxes (ZKT, KHRT) only generate on positive PKR profit; percentages pulled from `settings` at generation time.
4. Job number format: `{PREFIX}-{YEAR}-{SEQUENCE}` e.g. `EUMEX-2026-001`. Prefix editable in Settings. Sequence atomic and gap-free per year.
5. Jobs are archived, never hard-deleted, once financially active (paid rate or generated document exists).
6. `bl_data` holds only BL-specific fields — see Improvement #3 above.
7. One invoice per job by default; data model must not block multiple invoices per job in the future.

---

6. COMPLETE DATABASE SCHEMA

Use **better-sqlite3**. On connection: `PRAGMA foreign_keys = ON`, `PRAGMA journal_mode = WAL`.

Config tables

`settings`
```
key         TEXT PRIMARY KEY
value       TEXT NOT NULL
updated_at  DATETIME DEFAULT CURRENT_TIMESTAMP
```
Seed rows: `job_prefix` = `EUMEX`, `exchange_rate` = `280`, `zkt_percentage` = `2.5`, `khrt_percentage` = `7.5`.

`job_number_sequence`
```
year            INTEGER PRIMARY KEY
last_sequence   INTEGER NOT NULL DEFAULT 0
```

Master data tables

`clients`
```
```

```
id              INTEGER PRIMARY KEY AUTOINCREMENT
name            TEXT NOT NULL
type            TEXT NOT NULL CHECK(type IN ('SHIPPER','CONSIGNEE','NOTIFY','VENDOR'))
address         TEXT
phone           TEXT
email           TEXT
contact_person  TEXT
tax_id          TEXT
is_active       INTEGER NOT NULL DEFAULT 1 CHECK(is_active IN (0,1))
created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
updated_at      DATETIME DEFAULT CURRENT_TIMESTAMP
```

`ports`: `id`, `name` (NOT NULL), `code`, `country`, `is_active`, `created_at`, `updated_at`.

`commodities`: `id`, `name` (NOT NULL), `hs_code`, `description`, `is_active`, `created_at`, `updated_at`.

`container_types`: `id`, `code` (NOT NULL, UNIQUE), `description`, `weight_limit DECIMAL`, `is_active`, `created_at`, `updated_at`.

`jobs` (the hub)

```
id                    INTEGER PRIMARY KEY AUTOINCREMENT
job_number            TEXT UNIQUE NOT NULL
job_type              TEXT NOT NULL DEFAULT 'FCL' CHECK(job_type IN ('FCL','LCL'))
direction              TEXT NOT NULL DEFAULT 'EXPORT' CHECK(direction IN
('EXPORT','IMPORT'))
created_date          DATE DEFAULT CURRENT_DATE
shipper_id            INTEGER  REFERENCES clients(id)
consignee_id          INTEGER  REFERENCES clients(id)
notify_1_id           INTEGER  REFERENCES clients(id)
notify_2_id           INTEGER  REFERENCES clients(id)
commodity_id          INTEGER  REFERENCES commodities(id)
pol_id                INTEGER  REFERENCES ports(id)
pod_id                INTEGER  REFERENCES ports(id)
bl_number              TEXT
packages               INTEGER
gross_weight            DECIMAL
net_weight              DECIMAL
marks                  TEXT
```

```
fin_number               TEXT
etd                      DATE
eta                      DATE
status                   TEXT NOT NULL DEFAULT 'BOOKED' CHECK(status IN
('BOOKED','SAILED','DELIVERED','CLOSED','CANCELLED'))
notes                    TEXT
internal_notes           TEXT
exchange_rate_locked     DECIMAL
is_archived              INTEGER NOT NULL DEFAULT 0 CHECK(is_archived IN (0,1))
created_at               DATETIME DEFAULT CURRENT_TIMESTAMP
updated_at               DATETIME DEFAULT CURRENT_TIMESTAMP
```

Job child tables (all `ON DELETE CASCADE` from `jobs`)

`containers`: `id`, `job_id` (FK), `container_number`, `container_type_id` (FK), `seal_number`, `vessel`, `voyage`, `status CHECK(EMPTY/FULL/INTRANSIT/DELIVERED)`, `pickup_date`, `delivery_date`, `created_at`, `updated_at`.

`rates`: `id`, `job_id` (FK), `rate_type CHECK(BUYING/SELLING)`, `charge_type TEXT NOT NULL`, `amount DECIMAL NOT NULL CHECK(amount >= 0)`, `currency CHECK(USD/PKR) DEFAULT 'USD'`, `vendor_id` (FK → clients), `invoice_number`, `paid_status CHECK(UNPAID/PAID) DEFAULT 'UNPAID'`, `paid_date`, `created_at`, `updated_at`.

`taxes`: `id`, `job_id` (FK), `tax_type CHECK(ZKT/KHRT)`, `percentage DECIMAL NOT NULL`, `base_amount DECIMAL NOT NULL`, `amount DECIMAL NOT NULL`, `paid_status CHECK(UNPAID/PAID)`, `paid_date`, `paid_ref`, `created_at`, `updated_at`.

`documents`: `id`, `job_id` (FK), `doc_type CHECK(BL/INVOICE/BOOKING/CRO)`, `doc_number`, `file_path`, `generated_date DEFAULT CURRENT_DATE`, `sent_date`, `sent_to CHECK(CLIENT/CONSIGNEE/BANK/NULL)`, `sent_method CHECK(EMAIL/PRINT/COURIER/NULL)`, `created_at`.

`bl_data`: `id`, `job_id` (FK, UNIQUE — one BL record per job), `bl_number`, `vessel`, `voyage`, `port_loading`, `port_discharge`, `port_delivery`, `freight_terms CHECK(PREPAID/COLLECT/NULL)`, `free_days INTEGER`, `issued_date`, `created_at`, `updated_at`.

Indexes
`jobs(status)`, `jobs(shipper_id)`, `jobs(consignee_id)`, `jobs(pod_id)`, `jobs(pol_id)`, `jobs(is_archived)`, `jobs(created_date)`, `containers(job_id)`, `containers(container_type_id)`, `rates(job_id)`, `rates(vendor_id)`,

`rates(rate_type)`, `taxes(job_id)`, `documents(job_id)`, `bl_data(job_id)`,
`clients(type)`, `clients(is_active)`.

Triggers
`AFTER UPDATE` trigger on every table with `updated_at`, setting it to
`CURRENT_TIMESTAMP` on the affected row.

---

7. EXACT SEED DATA (use verbatim)

Clients (name, type, address, phone, email, contact_person, tax_id):
```
M.MUNIR AND SONS               | SHIPPER   | RAILWAY ROAD, RIYADH                    |
1234567          | info@mmunir.com     | Mr. Munir  | TAX-001
BANK ALFALAH                   | CONSIGNEE | BANK ROAD, LAHORE                       |
7654321          | info@bankalfalah.com| Mr. Ali    | TAX-002
DIHA AL MARIFA TRADING EST     | NOTIFY    | P.O.BOX 31799, ALKHOBAR 31952           |
+966500189429    | dm.ksa@yahoo.com    | Mr. Ahmed  | TAX-003
DURRA SABAH TRADING COMPANY    | NOTIFY    | AL SULAE AREA, RIYADH                   |
+966500189429    | dm.ksa@yahoo.com    | Mr. Khalid | TAX-004
MEEZAN BANK LIMITED            | CONSIGNEE | ZAHOOR ELAHI ROAD, LAHORE               |
1122334          | info@meezan.com     | Mr. Hassan | TAX-005
MEPCO CHEMICALS                | SHIPPER   | 16.5 KM SHEIKHU, JEDDAH                 |
5544332          | info@mepco.com      | Mr. Fahad  | TAX-006
ASIA ENTERPRISE                | SHIPPER   | CHAH BALANDAY, KARACHI                  |
6677889          | info@asia.com       | Mr. Imran  | TAX-007
KTM LEATHER                    | SHIPPER   | MEHR MANZIL, LC QINGDAO                 |
9988776          | info@ktm.com        | Mr. Raza   | TAX-008
TAQ ENTP CARGO SERVICES PVT LTD| VENDOR    | CARGO ROAD, KARACHI                     |
4433221          | info@taq.com        | Mr. Tariq  | TAX-009
H&H ENTERPRISES                | VENDOR    | ENTERPRISE ROAD, LAHORE                 |
5566778          | info@hh.com         | Mr. Hamid  | TAX-010
```

Ports (name, code, country):
```
JEDDAH   | JED | SAUDI ARABIA
DAMMAM   | DAM | SAUDI ARABIA
RIYADH   | RUH | SAUDI ARABIA
KARACHI  | KHI | PAKISTAN
QINGDAO  | QIN | CHINA
```

```
JEBELALI | JEA | UAE
HAMAD    | HAM | QATAR
```

Commodities (name, hs_code, description):
```
TENT      | 6306 | Tents and camping equipment
NIWAR     | 6306 | Canvas and ropes
CANVAS    | 6306 | Canvas rolls
LEATHER   | 4107 | Leather goods
CHEMICALS | 2800 | Industrial chemicals
```

Container types (code, description, weight_limit):
```
1x20    | 20-foot container    | 28000
1x40    | 40-foot container    | 30000
1x40HC  | 40-foot high cube    | 30000
20RFR   | 20-foot reefer       | 27000
40RFR   | 40-foot reefer       | 29000
```

Sample job `EUMEX-2026-001`:
- created_date: `2026-01-20`, shipper: M.MUNIR AND SONS, consignee: BANK ALFALAH, notify_1: DIHA AL MARIFA TRADING EST, notify_2: DURRA SABAH TRADING COMPANY, commodity: TENT, pod: JEDDAH
- bl_number: `078G100088`, packages: `193`, gross_weight: `15450`, net_weight: `15350`, marks: `193`, fin_number: `MBL-EXP-861186-12122025`, etd: `2026-01-25`, eta: `2026-02-10`, status: `BOOKED`

Container for this job: container_number `WHUS667790`, type `1x40` (40-foot container), vessel `CELSIUS EMMEN 20`, voyage `20`, status `FULL`.

Buying rates (charge_type, amount, currency, vendor = TAQ ENTP CARGO SERVICES PVT LTD):
```
FREIGHT        | 1110 | USD
PLACEMENT      | 100  | USD
LIFT ON        | 25   | USD
SEAL           | 7    | USD
BL CHARGES     | 85   | USD
CRO            | 3    | USD
```

```
SURRENDER/TLX  | 50   | USD
LIFT OFF       | 2700 | USD
```

Selling rates (charge_type, amount, currency, no vendor):
```
FREIGHT    | 1500 | USD
PLACEMENT  | 150  | USD
LIFT ON    | 40   | USD
SEAL       | 10   | USD
BL CHARGES | 120  | USD
CRO        | 5    | USD
```

---

8. BACKEND ARCHITECTURE

Strict one-way layering: `routes → controllers → services → repositories → db`.

```
src/
├── routes/          API endpoint wiring only, no logic
├── controllers/     Parse request, call service, shape response — no SQL, no math
├── services/
│   ├── profitService.js     currency-aware profit calculation
│   ├── taxService.js        ZKT/KHRT generation from settings
│   ├── jobNumberService.js  atomic job number generation
│   ├── documentService.js   PDF generation orchestration
│   └── backupService.js     SQLite backup rotation (used by Electron)
├── repositories/    All SQL lives here, one file per table, fully parameterized
├── middleware/
│   ├── errorHandler.js  centralized; typed errors → {success:false,error:{message,code}}; never leaks stack traces
│   ├── validate.js
│   └── logger.js
├── pdf/             blTemplate.js, invoiceTemplate.js, bookingTemplate.js, croTemplate.js
├── utils/           currency.js, dates.js, numbers.js
└── db/              connection.js, schema.js, seed.js
```

API surface (`{success, data, error}` envelope on every response):

```
GET/POST              /api/jobs
GET/PUT/DELETE        /api/jobs/:id
POST                  /api/jobs/:id/archive
GET/POST              /api/jobs/:id/containers
GET/POST              /api/jobs/:id/rates
POST                  /api/jobs/:id/generate-taxes
GET                   /api/jobs/:id/profit
GET/POST/PUT/DELETE   /api/clients, /api/ports, /api/commodities, /api/container-types
GET                   /api/reports/gpsht
GET                   /api/reports/jobgp
GET                   /api/reports/gpsht/export     (Excel)
GET                   /api/reports/jobgp/export
POST                  /api/documents/:jobId/bl
POST                  /api/documents/:jobId/invoice
POST                  /api/documents/:jobId/booking
POST                  /api/documents/:jobId/cro
GET/PUT               /api/settings
```

List endpoints support `?search=&status=&page=&limit=`.

Validation: required fields, enum values, numeric bounds (no negative amounts/weights) enforced both client- and server-side; FK existence validated before insert (clean 400, not a raw SQLite FK error).

Error handling: typed errors (`ValidationError`, `NotFoundError`, `ConflictError`) with attached status codes. Multi-table writes wrapped in transactions.

Coding standards: CommonJS, camelCase in JS / snake_case in DB, no magic numbers (tax %, exchange rate, job prefix always from `settings`), fully parameterized SQL only, nothing shipped as placeholder.

9. FRONTEND ARCHITECTURE

```
public/
├── index.html            Dashboard
├── new-job.html
├── job-detail.html
├── master-data.html
├── settings.html
```

```
├── reports-gpsht.html
├── reports-jobgp.html
├── css/style.css
└── js/
   ├── api.js         single fetch wrapper
   ├── table.js         reusable sortable/searchable/paginated table
   ├── modal.js
   ├── toast.js          notifications
   ├── validate.js       mirrors backend validation rules
   └── pages/            one controller per HTML page, no inline onclick=
```

Design reads as professional office software: clean, dense tables, color-coded status badges, fast load, minimal animation.

10. ELECTRON ARCHITECTURE

`main.js` spawns `server.js` as a child process on a fixed local port, health-checks it, opens a `BrowserWindow`. On close, `backupService` copies the SQLite file into `data/backups/` with a timestamp, keeping the last 30. Packaged via `electron-builder` (NSIS installer, desktop + start menu shortcuts, custom icon).

---

11. DEVELOPMENT ROADMAP (build in order, wait for my approval between phases)

1. **Database** — schema, indexes, triggers, seed data (verbatim from §7), repositories with transaction-safe helpers for job creation, profit calculation, tax generation, atomic job numbering.
2. **Backend** — routes, controllers, services, middleware, centralized error handling, logging.
3. **Frontend shell** — shared components (`api.js`, `table.js`, `modal.js`, `toast.js`, `validate.js`), base layout/CSS.
4. **Dashboard** — job list, search, filters, status badges, quick stats, recent jobs.
5. **Job Management** — create/edit/view/archive, containers, rates, live profit calculation, full validation.
6. **Master Data** — clients, ports, commodities, container types: full CRUD, search, active/inactive toggling.
7. **Documents** — PDF generation for BL, Invoice, Booking Note, CRO exactly per the field layouts in §12B, including the per-doc-type atomic numbering scheme and the `containers.pickup_location` column addition it requires.
8. **Reports** — GPSHT and JOBGP exactly per the column definitions and SQL in §12A, Excel export, filters, sorting, search, print.

9. **Electron packaging** — desktop wrapper, server bootstrap, backup rotation, installer, icons.
10. **QA pass** — bugs, security (SQL injection, input sanitization), performance, DB integrity, UI issues, API consistency, validation gaps. Fix everything before calling it production-ready.

---

12. STARTER CODE (already built — use as-is, do not regenerate from scratch)

The following files are already production-ready. Create them exactly as given, then continue building the remaining repositories/seed/services on top of them.

`package.json`

```json
{
 "name": "freightflow-pro",
 "version": "1.0.0",
 "description": "Freight Forwarding Management System",
 "main": "main.js",
 "scripts": {
   "start": "node server.js",
   "dev": "nodemon server.js",
   "test": "node tests/db.test.js",
   "db:init": "node src/db/schema.js",
   "db:seed": "node src/db/seed.js",
   "db:reset": "node src/db/reset.js",
   "electron": "electron .",
   "build": "electron-builder",
   "build:win": "electron-builder --win --x64"
 },
 "author": "",
 "license": "MIT",
 "dependencies": {
   "better-sqlite3": "^11.3.0",
   "express": "^4.18.2",
   "cors": "^2.8.5",
   "dotenv": "^16.0.3",
   "jspdf": "^2.5.1",
   "jspdf-autotable": "^3.5.31",
   "xlsx": "^0.18.5"
 },
```

```
 "devDependencies": {
   "electron": "^27.0.0",
   "electron-builder": "^24.6.4",
   "nodemon": "^2.0.22"
 },
 "build": {
   "appId": "com.freightflow.pro",
   "productName": "FreightFlow PRO",
   "directories": { "output": "dist" },
   "files": [
     "public/**/*", "src/**/*", "server.js", "main.js", "package.json", ".env"
   ],
   "extraResources": [ { "from": "data", "to": "data" } ],
   "win": { "target": "nsis", "icon": "icon.ico" },
   "nsis": {
     "oneClick": false,
     "allowToChangeInstallationDirectory": true,
     "createDesktopShortcut": true,
     "createStartMenuShortcut": true
   }
 }
}
```

`.env`
```
PORT=3000
DB_PATH=./data/freightflow.sqlite
NODE_ENV=development
```

`src/db/connection.js`
```javascript
// Single shared SQLite connection for the whole app. better-sqlite3 is
// synchronous, which matters here because several operations (creating a
// job with containers, generating taxes) must be atomic — a synchronous
// driver lets us write straightforward transactions without callback
// juggling that could hide partial writes.

const path = require('path');
const fs = require('fs');
const Database = require('better-sqlite3');
```

```
require('dotenv').config();

const dbPath = process.env.DB_PATH
 ? path.resolve(process.cwd(), process.env.DB_PATH)
 : path.join(__dirname, '..', '..', 'data', 'freightflow.sqlite');

const dataDir = path.dirname(dbPath);
if (!fs.existsSync(dataDir)) {
 fs.mkdirSync(dataDir, { recursive: true });
}

const db = new Database(dbPath);

db.pragma('foreign_keys = ON');
db.pragma('journal_mode = WAL');

module.exports = db;
```

`src/db/schema.js`
```javascript
// Creates every table, constraint, and index for FreightFlow PRO.
// Idempotent (CREATE TABLE IF NOT EXISTS) — safe to run on every boot.

const db = require('./connection');

function createSchema() {
 db.exec(`
   CREATE TABLE IF NOT EXISTS settings (
     key         TEXT PRIMARY KEY,
     value       TEXT NOT NULL,
     updated_at  DATETIME DEFAULT CURRENT_TIMESTAMP
   );

   CREATE TABLE IF NOT EXISTS job_number_sequence (
     year            INTEGER PRIMARY KEY,
     last_sequence   INTEGER NOT NULL DEFAULT 0
   );

   CREATE TABLE IF NOT EXISTS clients (
     id              INTEGER PRIMARY KEY AUTOINCREMENT,
     name            TEXT NOT NULL,
```

```
    type            TEXT NOT NULL CHECK(type IN
('SHIPPER','CONSIGNEE','NOTIFY','VENDOR')),
    address         TEXT,
    phone           TEXT,
    email           TEXT,
    contact_person  TEXT,
    tax_id          TEXT,
    is_active       INTEGER NOT NULL DEFAULT 1 CHECK(is_active IN (0,1)),
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at      DATETIME DEFAULT CURRENT_TIMESTAMP
  );

  CREATE TABLE IF NOT EXISTS ports (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    name        TEXT NOT NULL,
    code        TEXT,
    country     TEXT,
    is_active   INTEGER NOT NULL DEFAULT 1 CHECK(is_active IN (0,1)),
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME DEFAULT CURRENT_TIMESTAMP
  );

  CREATE TABLE IF NOT EXISTS commodities (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    name         TEXT NOT NULL,
    hs_code      TEXT,
    description  TEXT,
    is_active    INTEGER NOT NULL DEFAULT 1 CHECK(is_active IN (0,1)),
    created_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at   DATETIME DEFAULT CURRENT_TIMESTAMP
  );

  CREATE TABLE IF NOT EXISTS container_types (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    code          TEXT NOT NULL UNIQUE,
    description   TEXT,
    weight_limit  DECIMAL,
    is_active     INTEGER NOT NULL DEFAULT 1 CHECK(is_active IN (0,1)),
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at    DATETIME DEFAULT CURRENT_TIMESTAMP
  );
```

```
  CREATE TABLE IF NOT EXISTS jobs (
    id                    INTEGER PRIMARY KEY AUTOINCREMENT,
    job_number            TEXT UNIQUE NOT NULL,
    job_type              TEXT NOT NULL DEFAULT 'FCL' CHECK(job_type IN
('FCL','LCL')),
    direction              TEXT NOT NULL DEFAULT 'EXPORT' CHECK(direction IN
('EXPORT','IMPORT')),
    created_date          DATE DEFAULT CURRENT_DATE,
    shipper_id            INTEGER,
    consignee_id          INTEGER,
    notify_1_id           INTEGER,
    notify_2_id           INTEGER,
    commodity_id          INTEGER,
    pol_id                INTEGER,
    pod_id                INTEGER,
    bl_number              TEXT,
    packages               INTEGER,
    gross_weight            DECIMAL,
    net_weight              DECIMAL,
    marks                  TEXT,
    fin_number              TEXT,
    etd                    DATE,
    eta                    DATE,
    status                 TEXT NOT NULL DEFAULT 'BOOKED'
                            CHECK(status IN
('BOOKED','SAILED','DELIVERED','CLOSED','CANCELLED')),
    notes                   TEXT,
    internal_notes          TEXT,
    exchange_rate_locked    DECIMAL,
    is_archived             INTEGER NOT NULL DEFAULT 0 CHECK(is_archived IN (0,1)),
    created_at              DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at              DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (shipper_id)   REFERENCES clients(id),
    FOREIGN KEY (consignee_id) REFERENCES clients(id),
    FOREIGN KEY (notify_1_id)  REFERENCES clients(id),
    FOREIGN KEY (notify_2_id)  REFERENCES clients(id),
    FOREIGN KEY (commodity_id) REFERENCES commodities(id),
    FOREIGN KEY (pol_id)       REFERENCES ports(id),
    FOREIGN KEY (pod_id)       REFERENCES ports(id)
  );

  CREATE TABLE IF NOT EXISTS containers (
```

```
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    job_id              INTEGER NOT NULL,
    container_number    TEXT,
    container_type_id   INTEGER,
    seal_number         TEXT,
    vessel              TEXT,
    voyage              TEXT,
    status              TEXT NOT NULL DEFAULT 'EMPTY'
                         CHECK(status IN ('EMPTY','FULL','INTRANSIT','DELIVERED')),
    pickup_date         DATE,
    delivery_date       DATE,
    created_at          DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at          DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE,
    FOREIGN KEY (container_type_id) REFERENCES container_types(id)
  );

  CREATE TABLE IF NOT EXISTS rates (
    id               INTEGER PRIMARY KEY AUTOINCREMENT,
    job_id           INTEGER NOT NULL,
    rate_type        TEXT NOT NULL CHECK(rate_type IN ('BUYING','SELLING')),
    charge_type      TEXT NOT NULL,
    amount           DECIMAL NOT NULL CHECK(amount >= 0),
    currency         TEXT NOT NULL DEFAULT 'USD' CHECK(currency IN ('USD','PKR')),
    vendor_id        INTEGER,
    invoice_number   TEXT,
    paid_status      TEXT NOT NULL DEFAULT 'UNPAID' CHECK(paid_status IN
('UNPAID','PAID')),
    paid_date        DATE,
    created_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE,
    FOREIGN KEY (vendor_id) REFERENCES clients(id)
  );

  CREATE TABLE IF NOT EXISTS taxes (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    job_id         INTEGER NOT NULL,
    tax_type       TEXT NOT NULL CHECK(tax_type IN ('ZKT','KHRT')),
    percentage     DECIMAL NOT NULL,
    base_amount    DECIMAL NOT NULL,
    amount         DECIMAL NOT NULL,
```

```
    paid_status    TEXT NOT NULL DEFAULT 'UNPAID' CHECK(paid_status IN
('UNPAID','PAID')),
    paid_date      DATE,
    paid_ref       TEXT,
    created_at     DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at     DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE
  );

  CREATE TABLE IF NOT EXISTS documents (
    id               INTEGER PRIMARY KEY AUTOINCREMENT,
    job_id           INTEGER NOT NULL,
    doc_type         TEXT NOT NULL CHECK(doc_type IN
('BL','INVOICE','BOOKING','CRO')),
    doc_number       TEXT,
    file_path        TEXT,
    generated_date   DATE DEFAULT CURRENT_DATE,
    sent_date        DATE,
    sent_to          TEXT CHECK(sent_to IN ('CLIENT','CONSIGNEE','BANK') OR sent_to
IS NULL),
    sent_method      TEXT CHECK(sent_method IN ('EMAIL','PRINT','COURIER') OR
sent_method IS NULL),
    created_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE
  );

  CREATE TABLE IF NOT EXISTS bl_data (
    id               INTEGER PRIMARY KEY AUTOINCREMENT,
    job_id           INTEGER NOT NULL UNIQUE,
    bl_number        TEXT,
    vessel           TEXT,
    voyage           TEXT,
    port_loading     TEXT,
    port_discharge   TEXT,
    port_delivery    TEXT,
    freight_terms    TEXT CHECK(freight_terms IN ('PREPAID','COLLECT') OR
freight_terms IS NULL),
    free_days        INTEGER,
    issued_date      DATE,
    created_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE
```

```
  );

  CREATE INDEX IF NOT EXISTS idx_jobs_status        ON jobs(status);
  CREATE INDEX IF NOT EXISTS idx_jobs_shipper        ON jobs(shipper_id);
  CREATE INDEX IF NOT EXISTS idx_jobs_consignee      ON jobs(consignee_id);
  CREATE INDEX IF NOT EXISTS idx_jobs_pod            ON jobs(pod_id);
  CREATE INDEX IF NOT EXISTS idx_jobs_pol            ON jobs(pol_id);
  CREATE INDEX IF NOT EXISTS idx_jobs_archived       ON jobs(is_archived);
  CREATE INDEX IF NOT EXISTS idx_jobs_created_date   ON jobs(created_date);
  CREATE INDEX IF NOT EXISTS idx_containers_job      ON containers(job_id);
  CREATE INDEX IF NOT EXISTS idx_containers_type     ON
containers(container_type_id);
  CREATE INDEX IF NOT EXISTS idx_rates_job           ON rates(job_id);
  CREATE INDEX IF NOT EXISTS idx_rates_vendor        ON rates(vendor_id);
  CREATE INDEX IF NOT EXISTS idx_rates_type          ON rates(rate_type);
  CREATE INDEX IF NOT EXISTS idx_taxes_job           ON taxes(job_id);
  CREATE INDEX IF NOT EXISTS idx_documents_job       ON documents(job_id);
  CREATE INDEX IF NOT EXISTS idx_bl_data_job         ON bl_data(job_id);
  CREATE INDEX IF NOT EXISTS idx_clients_type        ON clients(type);
  CREATE INDEX IF NOT EXISTS idx_clients_active      ON clients(is_active);

  CREATE TRIGGER IF NOT EXISTS trg_clients_updated_at
    AFTER UPDATE ON clients BEGIN
      UPDATE clients SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
  CREATE TRIGGER IF NOT EXISTS trg_ports_updated_at
    AFTER UPDATE ON ports BEGIN
      UPDATE ports SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
  CREATE TRIGGER IF NOT EXISTS trg_commodities_updated_at
    AFTER UPDATE ON commodities BEGIN
      UPDATE commodities SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
  CREATE TRIGGER IF NOT EXISTS trg_container_types_updated_at
    AFTER UPDATE ON container_types BEGIN
      UPDATE container_types SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
  CREATE TRIGGER IF NOT EXISTS trg_jobs_updated_at
    AFTER UPDATE ON jobs BEGIN
      UPDATE jobs SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
  CREATE TRIGGER IF NOT EXISTS trg_containers_updated_at
```

```
    AFTER UPDATE ON containers BEGIN
      UPDATE containers SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
  CREATE TRIGGER IF NOT EXISTS trg_rates_updated_at
    AFTER UPDATE ON rates BEGIN
      UPDATE rates SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
  CREATE TRIGGER IF NOT EXISTS trg_taxes_updated_at
    AFTER UPDATE ON taxes BEGIN
      UPDATE taxes SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
  CREATE TRIGGER IF NOT EXISTS trg_bl_data_updated_at
    AFTER UPDATE ON bl_data BEGIN
      UPDATE bl_data SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
 `);
}

if (require.main === module) {
 createSchema();
 console.log('Schema created successfully.');
}

module.exports = { createSchema };
```

Not yet built (build these next, following the roadmap): `src/db/seed.js` (using the exact seed data in §7), all `src/repositories/*.js` files, `src/services/*.js`, `src/middleware/*.js`, `src/routes/*.js`, `server.js`, then proceed through Phases 2-10.

12A. REPORT DEFINITIONS (exact — do not redesign)

GPSHT Report (shipment/freight tracking)
Purpose: operational view of every shipment's tracking status — one row per container, not per job, since a job can have multiple containers.

```sql
SELECT
 j.job_number,
```

```
 s.name AS shipper,
 c.container_number,
 c.vessel,
 c.voyage,
 j.etd,
 j.eta,
 j.status,
 j.bl_number,
 p.name AS pod
FROM jobs j
LEFT JOIN clients s ON j.shipper_id = s.id
LEFT JOIN containers c ON j.id = c.job_id
LEFT JOIN ports p ON j.pod_id = p.id
WHERE j.is_archived = 0
ORDER BY j.etd DESC
```
Displayed columns, in order: Job Number, Shipper, Container Number, Vessel, Voyage, ETD, ETA, Status, BL Number, POD. Filterable by status, POD, ETD date range. Excel export uses the same column order and headers.

JOBGP Report (profit/tax analysis)
Purpose: management view of financial performance per job — one row per job.

For every non-archived job, compute:
```sql
-- Base job list
SELECT j.id, j.job_number, j.status, s.name AS shipper
FROM jobs j
LEFT JOIN clients s ON j.shipper_id = s.id
WHERE j.is_archived = 0
ORDER BY j.id DESC
```
Then for each job, using `profitService.calculateProfit(jobId)` and the job's `taxes` rows:

Column	Source
Job Number	`jobs.job_number`
Shipper	joined `clients.name`
Freight Received	`profit.total_selling` (USD, converted)
Freight Paid	`profit.total_buying` (USD, converted)
Profit (USD)	`profit.profit_usd`

```
| Profit (PKR) | `profit.profit_pkr` using `jobs.exchange_rate_locked` |
| ZKT | sum of `taxes.amount` where `tax_type='ZKT'` (0 if not yet generated) |
| KHRT | sum of `taxes.amount` where `tax_type='KHRT'` (0 if not yet generated) |
| Net GP | `profit_pkr - ZKT - KHRT` |
| Status | `jobs.status` |

Sort default: newest job first. Filterable by status and date range. Excel export mirrors the same column order and headers. Both reports must exclude archived jobs by default, with an optional "include archived" filter toggle.

---

## 12B. DOCUMENT TEMPLATES (Phase 7 — exact specification)

All four documents share a **company letterhead block** at the top: company name/logo placeholder, address, phone, email — pulled from `settings` (add `company_name`, `company_address`, `company_phone`, `company_email` as additional seeded settings keys). Every generated document gets a `documents` row recording its type, number, and generation date (per Business Rule §5/§7).

**Document numbering:** each doc type gets its own atomic per-year sequence, same pattern as job numbering (extend `job_number_sequence`-style logic into a generic `document_number_sequence` table keyed by `(doc_type, year)`), producing numbers like `BL-2026-001`, `INV-2026-001`, `BKG-2026-001`, `CRO-2026-001`. Never derive from `MAX(existing)`.

### Bill of Lading (BL)
Pulled from `jobs` + `clients` (shipper/consignee/notify) + `bl_data` (BL-specific overrides) + `containers`.
Layout sections, top to bottom:
1. Header: "BILL OF LADING", BL number, job number.
2. Shipper block (name, address) — from `jobs.shipper_id` → `clients`.
3. Consignee block — from `jobs.consignee_id`.
4. Notify Party 1 and 2 blocks — from `jobs.notify_1_id`/`notify_2_id`.
5. Vessel / Voyage, Port of Loading, Port of Discharge, Port of Delivery — from `bl_data`, falling back to `jobs.pol_id`/`pod_id` names if `bl_data` fields are blank.
6. Container & Seal table: container number, seal number, container type — from `containers`.
7. Marks & Numbers, Number of Packages, Description of Goods (commodity name + description), Gross Weight, Net Weight — from `jobs` + `commodities`.
8. Freight Terms (PREPAID/COLLECT), Free Days — from `bl_data`.
9. Footer: Place and Date of Issue, signature line.
```

```
### Invoice
Pulled from `jobs` + `clients` (bill-to party, typically shipper or consignee
depending on freight terms) + `rates` (SELLING rows only).
Layout:
1. Header: "INVOICE", invoice number, invoice date, job number reference.
2. Bill To block.
3. Shipment reference line: BL number, vessel/voyage, POL → POD, ETD.
4. Line-item table: one row per SELLING rate — Charge Type, Currency, Amount. Subtotal
per currency, since a job may have mixed-currency selling rates.
5. Total Due (shown per currency present; do not force-convert on the printed invoice
— conversion is for internal profit calculation only, not for what's billed to the
client).
6. Footer: payment terms/bank details placeholder (from `settings`), signature line.

### Booking Note
Pulled from `jobs` + `clients` (shipper) + `containers`.
Layout:
1. Header: "BOOKING NOTE", booking number, date, job number.
2. Shipper block.
3. Commodity, Packages, Gross Weight.
4. Port of Loading, Port of Discharge, ETD, ETA, Vessel, Voyage.
5. Container requirements: type(s) and quantity requested — from `containers` rows for
this job.
6. Special instructions — from `jobs.notes`.

### CRO Request (Container Release Order)
Pulled from `jobs` + `clients` (shipper) + `containers` + `container_types`.
Layout:
1. Header: "CRO REQUEST", CRO number, date, job number.
2. Requesting party (shipper) block.
3. Container Type, Quantity, Vessel, Voyage, Port of Loading.
4. Pickup location / empty depot — add `pickup_location TEXT` column to `containers`
(not present in the base schema in §6 — add it now).
5. Validity/free days — from `bl_data.free_days` if generated, otherwise blank for
manual entry.

**Note to builder:** these layouts define required fields and section order, not
pixel-level design. Use clean, bordered tables and a professional monospace/serif
document font (not a decorative UI font) so printed output reads like a real shipping
document, not a web page printout.
```

```
---

## 13. HOW TO PROCEED

1. Read this entire prompt fully before writing any code.
2. Confirm your understanding of the business workflow and this architecture back to me.
3. Flag anything you think is still missing or worth reconsidering.
4. Wait for my explicit approval.
5. Create the three starter files in §12 exactly as given.
6. Build `src/db/seed.js` using the exact seed data in §7, and the remaining repository files, explaining why each table/repository exists and how relationships work as you go.
7. Proceed through the remaining phases one at a time, only after I approve each one, following the module rules stated at the top of this prompt (explain → build → integrate cleanly → no placeholders).
```